endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 8

AGENDA CURS 8

1. Clase anonime
2. Interfete functionale
3. Expresii lambda
4. Method reference
5. Streams API

endava

CLASE ANONIME

- **Clasele anonime** sunt un tip special de clase interne care nu au un nume.
- Sunt folosite atunci cand ai nevoie de o implementare rapida a unei interfete sau a unei clase abstracte, de obicei pentru a trece o instanta ca argument sau pentru a redefini rapid comportamentul unei metode.
- Caracteristici principale:
  - Nu au nume propriu.
  - Se definesc si se instanțiaza intr-un singur pas.
  - Pot extinde o clasa sau implementa o interfata.
  - Pot accesa variabile finale sau efectiv finale din scope-ul exterior.
- Limitari:
  - Nu poti defini constructori expliciti.
  - Nu poti extinde mai multe clase sau implementa mai multe interfete in acelasi timp.
  - Codul poate deveni greu de citit daca se abuzeaza de clase anonime pentru implementari complexe.
- Cand sa folosesti clase anonime:
  - Cand implementarea este scurta si utilizata o singura data.
  - Cand vrei sa eviti definirea unei clase separate doar pentru o simpla suprascriere.

CLASE ANONIME

```java
abstract class Animal {
    abstract void makeSound();
}

Animal dog = new Animal() {
    @Override
    void makeSound() {
        System.out.println("Bark!");
    }
};
```

```java
Runnable task = new Runnable() {
    @Override
    public void run() {
        System.out.println("Task is running...");
    }
};
```


endava

INTERFETE FUNCTIONALE

- O **interfata functionala** este o interfata care contine **exact o metoda abstracta**.
- Sunt folosite mai ales in programarea functionala, in special cu **lambdas** si **method references**.
- Caracteristici principale:
  - Contine o singura metoda abstracta.
  - Poate avea metode default sau static, dar acestea nu afecteaza caracterul functional.
  - Se poate marca optional cu anotarea @FunctionalInterface, care ajuta la validare la compilare
- Cand sa folosesti interfete functionale:
  - Cand vrei sa transmiti comportamente ca argumente (ex: strategii, filtre, transformari).
  - Cand lucrezi cu librarii care accepta lambda expressions.

```java
@FunctionalInterface
interface MessagePrinter {
    void printMessage(String message);
}
```


endava

INTERFETE FUNCTIONALE

Interfata	Semnatura metodei	Descriere scurta
`Function<T, R>`	`R apply(T t)`	Transforma o valoare dintr-un tip `T` in alt tip `R`.
`BiFunction<T, U, R>`	`R apply(T t, U u)`	Combinare a doua valori `T` si `U` intr-un rezultat `R`.
`Consumer<T>`	`void accept(T t)`	Executa o actiune pe un obiect, fara a returna rezultat.
`BiConsumer<T, U>`	`void accept(T t, U u)`	Executa o actiune pe doua obiecte, fara rezultat.
`Supplier<T>`	`T get()`	Furnizeaza o valoare fara argumente; folosit pentru lazy init.
`Predicate<T>`	`boolean test(T t)`	Evalueaza o conditie booleana pe un obiect.
`BiPredicate<T, U>`	`boolean test(T t, U u)`	Evalueaza o conditie pe doua obiecte.
`UnaryOperator<T>`	`T apply(T t)`	Modifica o valoare, returnand aceeasi forma de tip.
`BinaryOperator<T>`	`T apply(T t1, T t2)`	Combina doua valori de acelasi tip si returneaza o valoare.

endava

EXPRESII LAMBDA

- O **expresie lambda** este o metoda anonima — adica un bloc de cod care poate fi tratat ca o valoare.
- Sunt folosite pentru a implementa **interfete functionale** intr-un mod concis.
- Sintaxa generala:
  - parameters: lista de parametri (pot fi omise tipurile daca pot fi deduse).
  - ->: operatorul "arrow" care separa parametrii de corpul functiei.
  - body: blocul de cod care este executat.
- Unde se folosesc expresiile lambda:
  - Pentru a implementa rapid interfete functionale.
  - In operatii pe colectii (ex: forEach, filter, map).
  - In APIs functionale (Stream API, CompletableFuture, Optional etc).
- Avantaje:
  - Cod mai scurt si mai clar.
  - Reduce nevoia de clase anonime separate.
  - Favorizeaza programarea functionala in Java.

EXPRESII LAMBDA

```java
@FunctionalInterface
interface GreetingService {
    void greet(String name);
}
```

```java
public class Main {
    public static void main(String[] args) {
        GreetingService greeting = (name) -> System.out.println("Hello, " + name + "!");

        greeting.greet("Alice");
        greeting.greet("Bob");
    }
}
```


endava

EXPRESII LAMBDA

Situație	Exemplu	Observatie
Un parametru, fara tip	`x -> x * x`	Paranteze optionale
Un parametru, cu tip	`(int x) -> x * x`	Paranteze obligatorii
Mai multi parametri	`(x, y) -> x + y`	Paranteze obligatorii
Un singur statement	`(x, y) -> x + y`	Fara acolade si return
Bloc de cod	`(x, y) -> { int r = x + y; return r; }`	Cu acolade si return

endava

METHOD REFERENCE

- **Method reference** este o forma mai concisa si mai clara de expresie lambda.
- **::** este operatorul de method reference.
- Cand sa folosesti method reference:
  - Cand expresia lambda doar apeleaza o metoda existenta, fara logica suplimentara.
  - Cand metoda pe care o invoci are aceeasi semnatura cu singura metoda abstracta a interfetei functionale pe care vrei o instantiezi.
  - Cand vrei sa ai codul mai clar si mai curat.

endava

METHOD REFERENCE

- Referinta la metoda de instanta a unui obiect

```
String str = "Hello";
Supplier<Integer> lengthSupplier = str::length;
System.out.println(lengthSupplier.get()); // Output: 5
```

- Referinta la metoda statica

```
Function<Double, Double> squareRoot = Math::sqrt;
System.out.println(squareRoot.apply(16.0)); // Output: 4.0
```

- Referinta la metoda de instanta a unui parametru

```
Function<String, Integer> stringLength = String::length;
System.out.println(stringLength.apply("Lambda")); // Output: 6
```

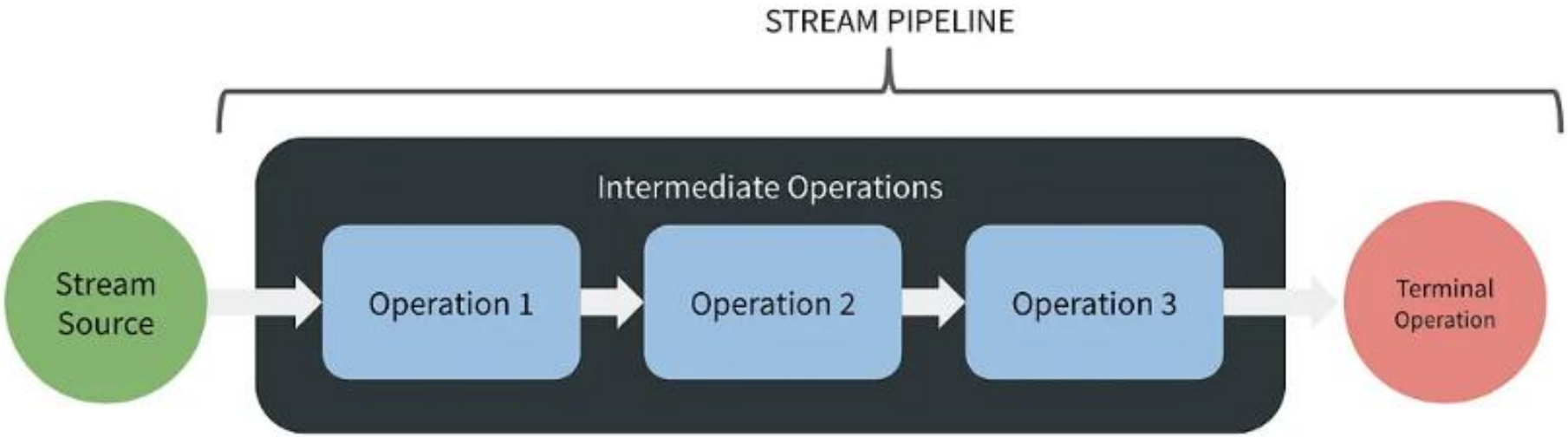
- Constructor reference

```
Supplier<List<String>> listSupplier = ArrayList::new;
List<String> list = listSupplier.get();
```

STREAMS API

- **Streams API** a fost introdus in Java 8 pentru a permite **procesarea functionala** a datelor intr-un mod declarativ, fluent si eficient.
- Stream-urile:
  - Nu stocheaza date (datele vin din colectii sau surse externe).
  - Nu modifica sursa initiala (sunt immutabile).
  - Suporta operatii lazy (executia reala se intampla doar cand este nevoie).
  - Pot fi secventiale sau paralele
- **Operatii intermediare:**
  - proceseaza datele dintr-un stream, returnand alt stream
  - Un stream poate avea zero, una sau oricat de multe operatii intermediare
- **Operatii terminale:**
  - incheie stream-ul si produc un rezultat sau un efect
  - un stream trebuie sa aiba exact o operatie terminala

STREAMS API


STREAM PIPELINE
Intermediate Operations
Stream Source
Operation 1
Operation 2
Operation 3
Terminal Operation

STREAMS API

Operatie intermediara	Descriere
`<R> Stream<T> filter(Predicate<? super T> predicate)`	Selecteaza elementele care indeplinesc o conditie
`<R> Stream<R> map(Function<? super T, ? extends R> mapper)`	Transforma fiecare element intr-un alt element
`<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)`	Descompune elemente in sub-streamuri si le aplatizeaza
`Stream<T> distinct()`	Elimina duplicatele
`Stream<T> sorted()`	Sorteaza elementele natural
`Stream<T> sorted(Comparator<? super T> comparator)`	Sorteaza elementele dupa un comparator specific
`Stream<T> limit(long maxSize)`	Pastreaza doar primele `n` elemente
`Stream<T> skip(long n)`	Sare peste primele `n` elemente
`Stream<T> peek(Consumer<? super T> action)`	Executa o actiune pe fiecare element, fara a modifica stream-ul

endava

STREAMS API

Operatie terminala	Descriere
`void forEach(Consumer<? super T> action)`	Executa o actiune pentru fiecare element din stream
`<R, A> R collect(Collector<? super T, A, R> collector)`	Colecteaza elementele intr-o structura (ex: lista, set, map)
`Optional<T> reduce(BinaryOperator<T> accumulator)`	Reduce stream-ul la un singur element folosind o operatie asociativa
`T reduce(T identity, BinaryOperator<T> accumulator)`	Reduce stream-ul cu o valoare initiala (identitate)
`long count()`	Returneaza numarul de elemente din stream
`Optional<T> min(Comparator<? super T> comparator)`	Returneaza elementul minim dupa un comparator
`Optional<T> max(Comparator<? super T> comparator)`	Returneaza elementul maxim dupa un comparator

endava

STREAMS API

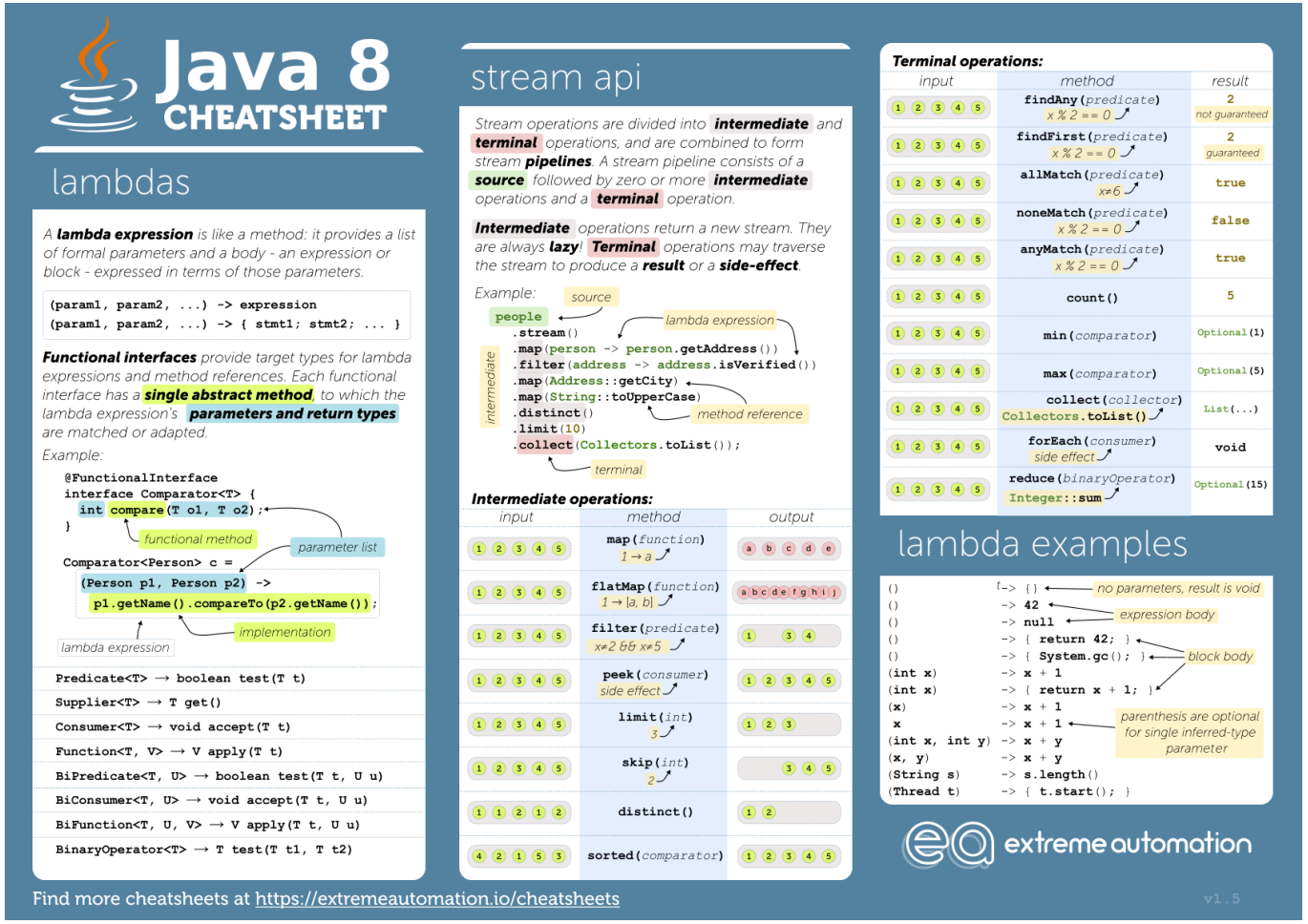
Operatie terminala	Descriere
`boolean anyMatch(Predicate<? super T> predicate)`	Returneaza true daca cel putin un element respecta conditia
`boolean allMatch(Predicate<? super T> predicate)`	Returneaza true daca toate elementele respecta conditia
`boolean noneMatch(Predicate<? super T> predicate)`	Returneaza true daca niciun element nu respecta conditia
`Optional<T> findFirst()`	Returneaza primul element (daca exista)
`Optional<T> findAny()`	Returneaza orice element (util mai ales in stream-uri paralele)

endava

STREAMS API

- **Parallel Stream** este o varianta de stream-ului care proceseaza elementele in paralel, folosind mai multe threads.
- Scopul este sa imbunatateasca performanta cand lucram cu volume mari de date.
- Caracteristici principale:
  - Imparte datele in subtask-uri si le proceseaza simultan
  - Ordinea de procesare nu este garantata (pentru operatii ca forEach)
  - Daca ordinea este importanta, se foloseste forEachOrdered()
  - Este mai eficient la colectii mari, dar poate fi mai lent pe date mici (din cauza costului de sincronizare)
- Cum se obtine un parallel stream:
  - Cu metoda parallelStream() din Collection
  - Cu metoda parallel() dintr-un stream existent

STREAMS API



Lambda and Streams API cheatsheet

endava

Q&A

THANK YOU

DANIELA SPILCĂ